

*Notes on Combinatorial and Probabilistic
Algorithms*
Riccardo Lo Iacono

July 16, 2026

Contents.

1	The analysis of Algorithms	1
1.1	Asymptotic notation	1
1.2	Amortized analysis	1
2	The dictionary problem	3
2.1	Red-Black trees	3

– 1 – The analysis of Algorithms.

As a good computer scientist knows, not all algorithms are the same nor are the resources these use; it is for these reasons that we need ways to analyze algorithms. In what follows, we first recall what asymptotic notation is, and then introduce the notion of *amortized analysis*.

– 1.1 – Asymptotic notation.

The idea behind asymptotic notation is that of analyzing algorithms, with respect to a given resource, more often than not being time, without caring about constants.

We recall the existence of three major notations: the *omega* notation (Ω), the *theta* notation (Θ) and the big-o notation ($\mathcal{O}$). Let $g(n)$ and $f(n)$ be two functions, then we define each notation as follows:

$$\Omega(f(n)) = \{g(n) \mid \exists c > 0, n_0 \geq 0 \text{ s.t. } g(n) \geq cf(n), \forall n \geq n_0\}$$

$$\Theta(f(n)) = \{g(n) \mid \exists c_1, c_2 > 0, n_0 \geq 0 \text{ s.t. } c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq n_0\}$$

$$\mathcal{O}(f(n)) = \{g(n) \mid \exists c, n_0 \geq 0 \text{ s.t. } g(n) \leq cf(n), \forall n \geq n_0\}$$

Although we give the definition in set notation, we write $g(n) = \mathcal{O}(f(n))$ to mean $g(n) \in \mathcal{O}(f(n))$; we do similarly for the other notations. Additionally, unless stated otherwise, assume we are interested in the time complexity of the algorithm.

– 1.2 – Amortized analysis.

Let us consider the following: assume we are given a data structure, and we are asked to compute a sequence of m operations on it; what is the overall complexity of all m operations, i.e., what is the time needed to compute all m operations.

It follows easily that, applying a worst case analysis, we need $m \cdot T_{MAX}(n)$, where $T_{MAX}(n)$ is the time complexity of the most expensive operation. For instance, under the worst case assumption, a sequence of m deletions from a tree has a complexity of $\mathcal{O}(m \cdot n)$.

We now introduce the notion of *amortized analysis*, and show how, over a sequence of operations, the real time complexity of an algorithm differs from the worst case one.

In its essence, amortized analysis does nothing more than averaging the complexity of all operations. That is, if $T_1(n)$ is the complexity of the first operation, $T_2(n)$ the one for the second operation, ..., $T_m(n)$ the complexity

Section 1 – The analysis of Algorithms

of the m -th operation, then the amortized time is given as

$$T_{\text{amortized}}(n) = \frac{1}{n} \sum_{i=1}^m T_i(n).$$

In other words, amortized time represents the time needed for each of the m operations.

The next section describes a simple ways to perform amortized analysis.

– 1.2.1 – Accounting method.

The core idea of the method follows that used by banks. In more details: to each operation we assign a cost, which represents its *amortized cost*. When the amortized cost of an operation exceeds its real cost, we are left with some credit that we can use to help pay other operations.

Example: Consider we need an algorithm to increment a n bits counter by an amount m . From a worst case analysis perspective, the algorithm needs a quadratic time, as the n -th bit may need to be flipped m times. It comes with no surprise that such a cost is well above the real one, as we are about to see.

Let us proceed this way: for each bit flip, assume that we are asked to pay a coin; then we proceed by paying two coins when flipping a bit to 1. The cost of the whole algorithm is easily seen to be given by the number of total bit flipped. But the following can be observed: the first increment, leaves us with a coin of credit, the second one does the same, etc. Thus, each operation requires a constant amount; that is, the amortized cost of the whole algorithm is $\mathcal{O}(n)$.

Remark: Other methods to perform amortized analysis exist, and their use varies with the algorithm we need to analyze. Though, all follow the same core idea: we overpay for some operations and use the credit thus produced to pay for others.

– 2 – The dictionary problem.

We turn our attention to one of the most common problem of computer science: the *dictionary problem*. In more details, we are given a set of elements S of some fixed universe U , and we are asked to design a data structure that allows to search, add and remove elements as efficiently as possible.

The problem has several solutions, each depending on the existence of a total order relation on the elements of S . Here we present Red-Black Trees (RB) as solution for the case in which a relation exists; while the notion of self-adjusting data structure is explored for case in which no such relation is available.

– 2.1 – Red-Black trees.

Red-Black trees are a special type of Binary Search Trees (BST), from which they differ for the following properties:

- Each node of a RB tree is either red or black;
- The root of a RB tree is always black;
- Each leaf is black;
- Each red node has only black children; and
- For each node, all simple path from the node to a descendant leaf, contains the same number of black nodes.

For any RB tree T , the following result holds.

Lemma: *If T is a Red-Black tree with n internal nodes, then T has height at $2\log(n + 1)$.*

The proof of the above Lemma is based on the notion of black height of a RB tree, which essentially represents the number of black nodes in the path from a node (the proof considers the root) to any descendant leaf.

In section 2.1.1 we describe the rotation operations needed to implement the dictionary operations, later described in Sections 2.1.2 and 2.1.3.

– 2.1.1 – Rotations.

As easily understood, RB trees require additional work to ensure that each property holds. In fact, differently from BSTs, the insertion and/or deletion of a node may cause one or more property to break, as discussed in more detail in Sections 2.1.2 and 2.1.3.

To fix these issue, some auxiliary functions need to be defined; these in turns make use of *rotation* which are local operations on pointers. Rotations

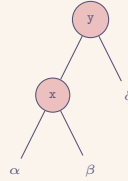


Figure 1: Example of a RB tree.

are essentially of two types: left-to-right (or simply left-rotation) and right-to-left (or simply right rotation). We simply consider right rotations, as left rotations are symmetric.

Consider the sample tree shown in Figure 1. Assume that after some operations we need to apply a right rotation on node x . Then we proceed as follow: we let y point to be x left children, we link x 's parent to y and move x to y 's right. In Procedure Rotate-right we provide the pseudo code to implement RB's right rotation.

Procedure Rotate-right(T, x)
<pre> $y = x.left$; $x.left = y.right$ if $y.right \neq \text{nil}$ then $y.right.p = x$ $y.p = x.p$ if $x.p == \text{nil}$ then $T.root = y$ else if $x == x.p.right$ then $x.p.right = y$ else $x.p.left = y$ $x.right = x$ $x.p = y$ end </pre>

– 2.1.2 – Insert.

Inserting a node in a RB tree works similarly to that in a BST, with a couple of differences. First, we color the node just added red, then, to ensure all RB properties hold a auxiliary procedure is executed. Procedure RB-Insert is the pseudo code used to implement insertion in a red-black tree, while Procedure Insert-fixup the pseudo code for the auxiliary function is given.

Before analyzing how Procedure Insert-fixup works, let us see which of the

Section 2 – The dictionary problem

RB properties may be violated. Properties 1, 3, 5 still hold¹. On the other hand, properties 2 and 4 are not guaranteed to hold; the former failing when the new node becomes the root, the latter if the parent is also red.

Procedure RB-Insert(T, z)
<pre><i>y</i> = nil ; <i>x</i> = <i>T.root</i> while <i>x</i> ≠ nil do <i>y</i> = <i>x</i> if <i>z.key</i> < <i>x.key</i> then <i>x</i> = <i>x.left</i> else <i>x</i> = <i>x.right</i> end <i>z.p</i> = <i>y</i> if <i>y</i> == nil then <i>T.root</i> = <i>z</i> else if <i>z.key</i> < <i>y.key</i> then <i>y.left</i> = <i>z</i> else <i>y.right</i> = <i>z</i> <i>z.left</i> = nil <i>z.right</i> = nil <i>z.color</i> = “Red” Insert-fixup(<i>T, z</i>)</pre>

The analysis of Procedure Insert-fixup can be broke down in six cases, pairwise symmetric. We consider for this reason case 1-3. Let z be the inserted node, let $z.p$ denote z 's parent and let y denote z 's uncle.

Case 1: both $z.p$ and y are red. Since both $z.p$ and y are red, and we can assume $z.p.p$ to exist and being black, we simply recolor the nodes, locally restoring property 4. Then, we call the auxiliary procedure on $z.p.p$.

Case 2: y is black and z is a right child. We apply a left rotation on $z.p$ falling this way in case 3.

Case 3: y is black and z is a left child. Since $z.p.p$ was black prior to the insertion, we can simply apply some recoloring a right rotation on $z.p.p$.

Once the loop has finished the only property that may still fail is property 2, which is fixed by recoloring the root black.

¹We assume that the right and left children of the inserted node are set to **nil**, which is assumed to have a black color.

Procedure Insert-fixup(T, z)

```

while  $z.p.color == \text{“Red”}$  do
  if  $z.p == z.p.p.left$  then
     $y = z.p.p.right$  ;
    if  $y.color == \text{“Red”}$  then
       $z.p.color = \text{“Black”}$ 
       $y.color = \text{“Black”}$ 
       $z.p.p.color = \text{“Red”}$ 
       $z = z.p.p$ 
    else if  $z == z.p.right$  then
       $z = z.p$ 
      Rotate-left( $T, z$ )
       $z.p.color = \text{“Black”}$ 
       $z.p.p.color = \text{“Red”}$ 
      Rotate-right( $T, z.p.p$ )
    else
       $y = z.p.p.left$  ;
      if  $y.color == \text{“Red”}$  then
         $z.p.color = \text{“Black”}$ 
         $y.color = \text{“Black”}$ 
         $z.p.p.color = \text{“Red”}$ 
         $z = z.p.p$ 
      else if  $z == z.p.left$  then
         $z = z.p$ 
        Rotate-right( $T, z$ )
         $z.p.color = \text{“Black”}$ 
         $z.p.p.color = \text{“Red”}$ 
        Rotate-left( $T, z.p.p$ )
  end
 $T.root.color = \text{“Black”}$ 

```

Section 2 – The dictionary problem

– 2.1.3 – Delete.

Similarly for insertion, deleting a node from a red-black tree works as for the BSTs, with a few differences depending on the original position of the node to remove. As before, we make use of an auxiliary procedure to fix any violations. In Procedure RB-delete the pseudo code to implement deletion is shown, while in Procedure RB-delete-fixup we show the pseudo code for the auxiliary procedure.

Let us consider which properties may be violated: first, property 2 fails to hold when the node to remove was the root and the new root is black, Property 4 fails when both the replacing node and its right child are red; lastly, properties 1 and 5 fail when nodes are moved around.

Procedure RB-delete(T, z) $y = z$ $y - \text{starting} - \text{color} = y.\text{color}$ if $z.\text{left} == \text{nil}$ then $x = z.\text{right}$ RB-transplant($T, z, z.\text{right}$) else if $z.\text{right} == \text{nil}$ then $x = z.\text{left}$ RB-transplant($T, z, z.\text{left}$) else $y = \text{Tree} - \text{Min}(z.\text{right})$ $x = y.\text{right}$ if $y.p == z$ then $x.p = z$ else RB-transplant($T, y, y.\text{right}$) $y.\text{right} = z.\text{right}$ $y.\text{right}.p = y$ RB-transplant(T, z, y) $y.\text{left} = z.\text{left}$ $y.\text{left}.p = y$ $y.\text{color} = z.\text{color}$ if $y - \text{starting} - \text{color} == \text{"Black"}$ then RB-delete-fixup(T, x)

We can analyze Procedure RB-delete-fixup as eight cases, pairwise symmetric. We consider for this reason only case 1-4. To ease our explanation, let z be the node to remove, $z.p$ its parent, y the minimum in the z 's right sub-tree and let x be y 's right child.

Section 2 – The dictionary problem

Case 1: x 's sibling w is “Red”. We know that w must have a black child, thus a rotation on $x.p$ will solve the violation.

Case 2: w is “Black” and so are its children. Since both x and w are black, we simply “push” the blackness upward, and repeat the procedure on the parent.

Case 3: w is “Black”, its left child is “Red” and the left child is “Red”.
A simple color exchange between w and $w.left$ followed by a right rotation, fixes the issue.

Case 4: w is “Black” and its right child is “Red”. Some color exchanges and a left rotation on $x.p$ will solve the violations.

As for insertion, once the loop has finished the only property that may still fail is property 2, which is fixed by recoloring the root.

Procedure RB-delete-fixup(T, x)

```
while  $x \neq T.root$  and  $x.color \neq \text{“Black”}$  do
  if  $x == x.p.left$  then
     $w = x.p.right$ 
    if  $w.color == \text{“Red”}$  then
       $w.color = \text{“Black”}$ 
       $x.p.color = \text{“Red”}$ 
      Rotate-left( $T, x.p$ )
    if  $w.left.color == \text{“Black”}$  and  $w.right.color == \text{“Black”}$ 
      then
         $w.color = \text{“Red”}$ 
         $x = x.p$ 
      else if  $w.right.color == \text{“Red”}$  then
         $w.left.color = \text{“Black”}$ 
         $w.color = \text{“Red”}$ 
        Rotate-right( $T, w$ )
         $w = x.p.left$ 
         $w.color = x.p.color$ 
         $x.p.color = \text{“Black”}$ 
        Rotate-left( $T, x.p$ )
         $x = T.root$ 
      else
        ( same as above with “right” and “left” exchanged )
  end
   $x.color = \text{“Black”}$ 
```
